C L O U D S E C U R I T Y

Automazione e Hardening con Ansible

in ambiente Cloud

Alberto Cirillo

Cybersecurity – Cloud Security

Infobasic - Pescara

Anno Accademico 2025/2026

Indice

1. Introduzione ad Ansible.....
1.1 Che cos'e' Ansible.....
1.2 Perche' Ansible nella Cloud Security.....
1.3 Caratteristiche principali.....
1.4 Architettura di Ansible.....

2. Ambiente di laboratorio.....
2.1 Scelta del provider cloud.....
2.2 Architettura del laboratorio.....
2.3 Configurazione della rete cloud.....

3. Configurazione dell'accesso SSH.....
3.1 Generazione delle chiavi.....
3.2 Bootstrap e rotazione delle credenziali.....

4. Installazione e configurazione di Ansible.....
4.1 Installazione.....
4.2 File di configurazione (ansible.cfg).....
4.3 File di inventario (inventory.ini).....
4.4 Test di connettivita'.....

5. Stato iniziale dei server.....
5.1 Comandi di verifica.....
5.2 Riepilogo stato iniziale.....

6. Playbook di hardening.....
6.1 Panoramica.....
6.2 Contenuto del playbook.....
6.3 Analisi dettagliata dei task.....
6.4 Esecuzione e risultati.....

7. Verifiche post-hardening.....
7.1 Configurazione SSH.....
7.2 Stato del firewall.....
7.3 Stato di fail2ban.....
7.4 Fail2ban in azione.....
7.5 Idempotenza in pratica.....

8. Playbook di reset.....
8.1 Scopo.....
8.2 Contenuto del playbook.....
8.3 Verifiche post Reset.....
8.4 Re-Hardening.....

9. Conclusioni.....

1. Introduzione ad Ansible

L'obiettivo di questo progetto è dimostrare come sia possibile automatizzare la configurazione sicura di server cloud, riducendo gli errori umani e garantendo coerenza tra i sistemi.

1.1 Che cos'e' Ansible

Ansible e' uno strumento open source per l'automazione IT sviluppato da Red Hat e consente di automatizzare attivita' come la configurazione dei sistemi, il deployment di applicazioni, l'orchestrazione di infrastrutture e il provisioning di risorse cloud. A differenza di altri strumenti simili, Ansible non richiede l'installazione di alcun software aggiuntivo sulle macchine gestite: opera in modalita' agentless, utilizzando il protocollo SSH per comunicare con i nodi remoti.

Il suo funzionamento si basa su un approccio dichiarativo: l'amministratore descrive lo stato desiderato del sistema attraverso file di configurazione scritti in YAML, e Ansible si occupa di portare ogni macchina a quello stato. Questo approccio rende l'automazione prevedibile, ripetibile e facile da comprendere anche per chi non ha esperienza con strumenti di Configuration Management.

1.2 Perche' Ansible nella Cloud Security

Nel contesto della sicurezza in ambiente cloud, la gestione manuale delle configurazioni rappresenta un rischio significativo. Errori umani, configurazioni inconsistenti tra server diversi e la difficoltà di mantenere una baseline di sicurezza uniforme sono problemi comuni nelle infrastrutture gestite manualmente. In particolare, le *misconfiguration* rappresentano una delle principali cause di incidenti di sicurezza in ambiente cloud.

Ansible risolve questi problemi permettendo di definire l'infrastruttura e le sue configurazioni come codice. Questo approccio è noto come *Infrastructure as Code (IaC)*: anziché accedere a ogni server e configurarlo manualmente tramite terminale, l'amministratore descrive lo stato desiderato dell'infrastruttura attraverso file di configurazione, e Ansible si occupa di applicarlo automaticamente.

Il file diventa quindi la "fonte di verità" dell'infrastruttura: è versionabile, condivisibile, riutilizzabile e ripetibile, riducendo drasticamente il rischio di errori umani.

Quando questo principio viene applicato specificamente alle configurazioni di sicurezza, si parla di *Security as Code*: le misure di protezione (regole firewall, configurazioni SSH, sistemi di prevenzione delle intrusioni) vengono definite in file di codice e applicate automaticamente in modo uniforme su tutti i server.

Questo garantisce che nessun sistema venga configurato in modo differente o dimenticato, e che le policy di sicurezza siano sempre documentate, tracciabili e verificabili. Il playbook di hardening realizzato in questo progetto rappresenta un esempio concreto di questo approccio.

1.3 Caratteristiche principali

Agentless

Ansible è sviluppato in Python e utilizza moduli scritti in questo linguaggio per eseguire le operazioni sui nodi remoti: ad ogni task, Ansible trasferisce via SSH il modulo necessario sul nodo, lo esegue, raccoglie il risultato e lo rimuove automaticamente. Per questo motivo, Python deve essere presente sulle macchine gestite, pur non trattandosi di un agente permanente.
L'utente non interagisce direttamente con Python, ma con file YAML, mentre Python rappresenta il motore di esecuzione interno.

Linguaggio YAML

Le configurazioni di Ansible sono scritte in YAML, un formato di serializzazione dati leggibile dall'uomo. È importante distinguere i due linguaggi coinvolti: YAML è il formato con cui l'amministratore descrive le operazioni da eseguire, mentre Python è il linguaggio con cui Ansible le interpreta e le esegue concretamente sulle macchine.

YAML e' intuitivo e non richiede competenze di programmazione per essere compreso. Questo rende i playbook Ansible auto-documentanti e facilmente revisionabili.

Ecco un esempio di task scritto in YAML:

```
- name: Aggiornamento dei pacchetti di sistema
  apt:
    update_cache: yes --> Aggiorna la lista dei pacchetti disponibili (sudo apt update)
    upgrade: dist --> Installa tutti gli aggiornamenti disponibili (sudo apt upgrade)
```

Come si puo' notare, il codice e' leggibile anche da chi non ha familiarita' con lo strumento: il task ha un nome descrittivo, utilizza il modulo apt e specifica due parametri (aggiorna la cache dei pacchetti e applica tutti gli aggiornamenti disponibili).

Idempotenza

L'idempotenza e' un concetto fondamentale di Ansible: ogni task verifica lo stato attuale del sistema prima di agire. Se il sistema e' gia' nello stato desiderato, Ansible non apporta modifiche e restituisce lo stato "ok". Se invece e' necessaria una modifica, Ansible la applica e restituisce lo stato "changed". Questo garantisce che un playbook possa essere eseguito piu' volte senza effetti collaterali indesiderati.
Questo consente anche di utilizzare i playbook anche come strumento di verifica dello stato di sicurezza del sistema.

Approccio Push-based

Ansible utilizza un modello push: il control node si connette attivamente ai managed nodes e invia i comandi da eseguire. Questo si contrappone al modello pull, dove sono i nodi gestiti a contattare un server centrale per ottenere le configurazioni. L'approccio push offre un controllo immediato e diretto sull'esecuzione delle attivita'.

1.4 Architettura di Ansible

L'architettura di Ansible si compone di diversi elementi che collaborano tra loro:

Control Node

E' la macchina su cui Ansible e' installato e da cui vengono eseguite tutte le operazioni. Il control node contiene i playbook, i file di inventario, le chiavi SSH e il file di configurazione. (La configurazione operativa di Ansible (file ansible.cfg) verrà descritta nel dettaglio più avanti). Il Control Node puo' essere qualsiasi macchina Linux con Python e Ansible installati.

Managed Nodes

Sono le macchine remote che vengono gestite e configurate da Ansible. Non necessitano di alcun software aggiuntivo oltre a SSH e Python. Possono essere server fisici, macchine virtuali locali o istanze cloud.

Inventario (Inventory)

L'inventario e' un file che definisce l'elenco dei managed nodes e li organizza in gruppi logici. Puo' essere un semplice file INI, YAML statico, oppure un inventario dinamico che interroga provider cloud per ottenere l'elenco delle macchine in tempo reale.

Esempio di inventario:

```
[servers]
server-1 ansible_host=192.168.1.10
server-2 ansible_host=192.168.1.11
```

Playbook

Un playbook e' un file YAML che descrive una serie di task da eseguire sui managed nodes. Ogni task utilizza un modulo Ansible specifico per svolgere un'operazione.
Il playbook utilizzato in questo progetto verrà mostrato nel dettaglio più in avanti.

Moduli

I moduli sono le unita' operative di Ansible. Ogni modulo e' specializzato in un'attivita' specifica. Ansible include centinaia di moduli predefiniti.

Ecco alcuni esempi dei moduli utilizzati in questo progetto:

```
# Modulo apt - gestione pacchetti su sistemi Debian/Ubuntu
- name: Installa il pacchetto UFW
  apt:
    name: ufw
    state: present

# Modulo lineinfile - modifica una riga in un file di configurazione
- name: Disabilita il login root via SSH
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?PermitRootLogin'
```

```
    line: 'PermitRootLogin no'


# Modulo service - gestione dei servizi di sistema
- name: Riavvia il servizio SSH
  service:
    name: ssh
    state: restarted

# Modulo ufw - gestione del firewall
- name: Consenti traffico SSH
  ufw:
    rule: allow
    port: '22'
    proto: tcp
```

Handlers

Gli handler sono task speciali che vengono eseguiti solo quando notificati da un altro task che ha effettuato una modifica. Questo evita operazioni inutili.
Questo meccanismo contribuisce ulteriormente all'idempotenza del playbook, evitando operazioni ridondanti.

Ad esempio, il riavvio di un servizio viene effettuato solo se la sua configurazione e' stata effettivamente modificata:

```
# Nei task: la clausola 'notify' indica quale handler attivare
- name: Modifica configurazione SSH
  lineinfile:
    path: /etc/ssh/sshd_config
    regexp: '^#?PermitRootLogin'
    line: 'PermitRootLogin no'
  notify: Riavvio SSH        --> attiva l'handler solo se il file viene modificato

# Nella sezione handlers: il task viene eseguito solo se notificato
handlers:
  - name: Riavvio SSH        --> il nome deve corrispondere al notify
    service:
      name: ssh
      state: restarted
```

2. Ambiente di laboratorio

2.1 Scelta del provider cloud

Per la realizzazione del laboratorio e' stato scelto Oracle Cloud Infrastructure come provider cloud, in particolare il tier Always Free.

Questa scelta è motivata da diversi fattori:

- le risorse Always Free sono disponibili senza limiti di tempo e senza costi
- permettono di mantenere l'ambiente attivo sia durante lo sviluppo che durante l'esposizione
- offrono risorse sufficienti per il laboratorio
- la presenza della regione Milano consente una bassa latenza dall'Italia

2.2 Architettura del laboratorio

L'architettura del laboratorio e' composta da tre macchine:

Control Node: una macchina virtuale Ubuntu installata su VMware Workstation. Questa macchina è il punto di controllo centrale dell'infrastruttura, ospitando Ansible e tutti i file di configurazione. E' la macchina dalla quale vengono impartiti i comandi a tutti i nodi gestiti.

Managed Node 1: istanza Ubuntu 22.04 Minimal su Oracle Cloud

Managed Node 2: istanza Ubuntu 22.04 Minimal su Oracle Cloud

I managed nodes rappresentano le macchine remote gestite da Ansible: ricevono i comandi dal control node ed eseguono le operazioni definite nei playbook.

Entrambi i managed nodes sono stati configurati con le risorse previste dal tier Always Free di Oracle Cloud (1 CPU e 1 GB di RAM). Si tratta di risorse limitate, ma più che sufficienti per le operazioni di hardening previste dal progetto.

☐	Nome	Stato	IP pubblico	IP privato	Forma	Conteggio OCPU	Memoria (GB)
☐	managed-node-2 Sempre gratis	In esecuzione	130.110.1.219	10.0.0.106	VM.Standard.E2.1.Micro	1	1
☐	managed-node-1 Sempre gratis	In esecuzione	92.4.160.80	10.0.0.187	VM.Standard.E2.1.Micro	1	1

2.3 Configurazione della rete cloud

Per consentire la comunicazione tra il control node e i managed nodes, è stata configurata una Virtual Cloud Network (VCN) su Oracle Cloud. Una VCN è una rete virtuale privata all'interno del cloud, equivalente a una rete locale tradizionale ma interamente definita via software. Tutte le risorse cloud del progetto (le due istanze) risiedono all'interno di questa rete.

VCN: ansible-vnc, con blocco di indirizzi 10.0.0.0/16. La notazione /16 indica che all'interno di questa rete virtuale sono disponibili tutti gli indirizzi compresi tra 10.0.0.0 e 10.0.255.255, per un totale di oltre 65.000 indirizzi utilizzabili.
La VCN definisce lo spazio di indirizzi complessivo, all'interno del quale vengono create le subnet

come suddivisioni più piccole, ciascuna con il proprio blocco di indirizzi e le proprie regole di accesso (pubblica e privata).

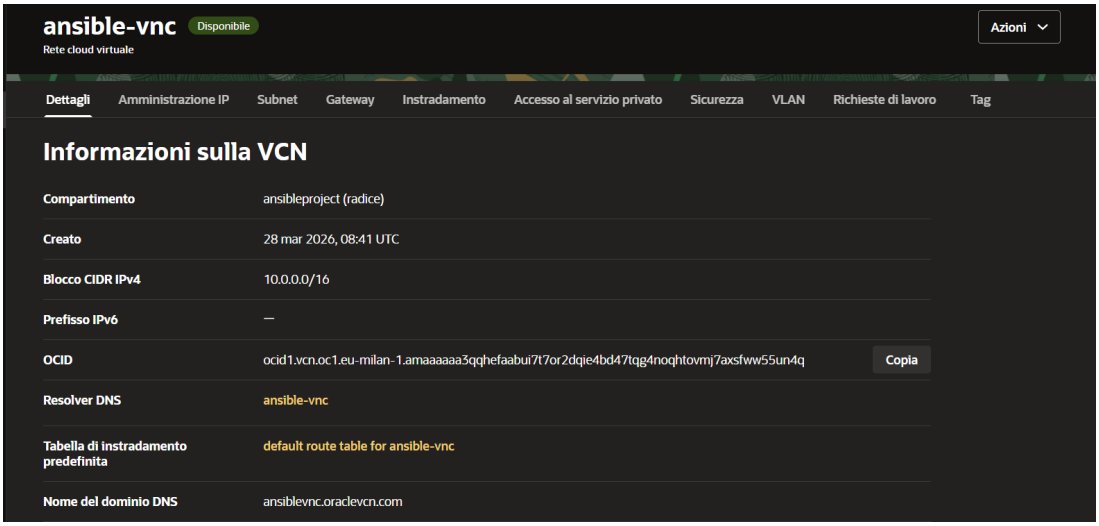
Subnet pubblica: 10.0.0.0/24, 256 indirizzi IP disponibili. Questa subnet è di tipo pubblico, il che significa che le istanze al suo interno possono ricevere un indirizzo IP pubblico e sono raggiungibili da internet. Le due istanze del laboratorio si trovano in questa subnet.

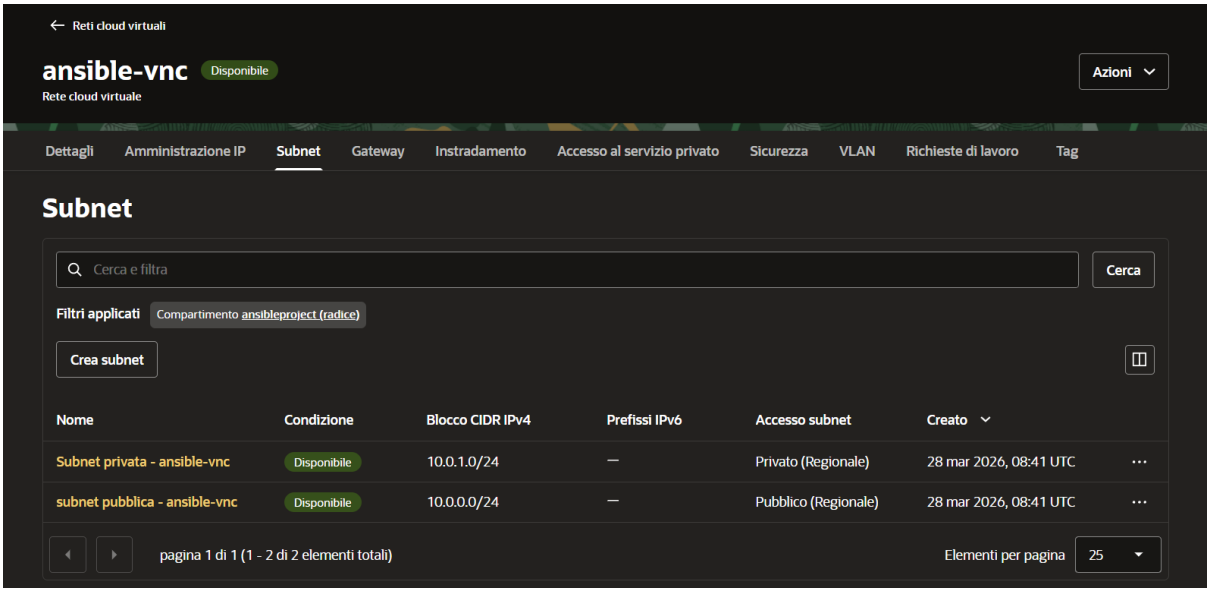
Subnet privata: 10.0.1.0/24, 256 indirizzi IP disponibili. Questa subnet viene creata automaticamente dal wizard di Oracle Cloud durante la configurazione della VCN. Le risorse in una subnet privata non ricevono un indirizzo IP pubblico e non sono raggiungibili direttamente da internet. Non è utilizzata in questo progetto, ma in un'architettura di produzione verrebbe impiegata per ospitare servizi interni come database o applicazioni backend che non devono essere esposti all'esterno.

Security List: ad ogni subnet è associata una Security List, che funziona come un firewall a livello di rete virtuale. La Default Security List, creata automaticamente da Oracle Cloud durante il provisioning della VCN, include già due regole predefinite:

- una regola di ingresso che consente il traffico TCP sulla porta 22 (SSH) da qualsiasi indirizzo IP sorgente (0.0.0.0/0), permettendo così la connessione SSH da qualsiasi host (ma l'autenticazione effettiva avviene a livello SSH tramite chiave crittografica.)
  Questa configurazione è coerente con il nostro progetto, in quanto Ansible utilizza esclusivamente il protocollo SSH per comunicare con i managed nodes e non necessita di altre porte aperte
- una regola di uscita che consente tutto il traffico in uscita verso qualsiasi destinazione (0.0.0.0/0) su tutti i protocolli, necessaria affinché le istanze possano scaricare aggiornamenti, pacchetti e comunicare con servizi esterni

Ad ogni istanza è stato assegnato un indirizzo IPv4 pubblico, necessario per rendere le macchine raggiungibili dal control node attraverso internet e permettere ad Ansible di connettersi ai nodi.




Reti cloud virtuali
ansible-vnc
Disponibile
Rete cloud virtuale
Azioni
Dettagli
Amministrazione IP
Subnet
Gateway
Instradamento
Accesso al servizio privato
Sicurezza
VLAN
Richieste di lavoro
Tag
Subnet
Cerca e filtra
Cerca
Filtri applicati
Compartimento ansibleproject (radice)
Crea subnet
Nome
Condizione
Blocco CIDR IPv4
Prefissi IPv6
Accesso subnet
Creato
Subnet privata - ansible-vnc
Disponibile
10.0.1.0/24
—
Privato (Regionale)
28 mar 2026, 08:41 UTC
subnet pubblica - ansible-vnc
Disponibile
10.0.0.0/24
—
Pubblico (Regionale)
28 mar 2026, 08:41 UTC
pagina 1 di 1 (1 - 2 di 2 elementi totali)
Elementi per pagina
25

Default Security List for ansible-vnc
Disponibile
Lista di sicurezza
Azioni
Il traffico dell'istanza è controllato dalle regole firewall di ogni istanza, oltre che da questa lista di sicurezza
Dettagli
Regole di sicurezza
Tag
Regole di entrata
Cerca e filtra
Cerca
Aggiungi regole di entrata
Azioni
Senza conservazione dello stato
Origine
Protocollo IP
Intervallo porte di origine
Intervallo di porte di destinazione
Tipo e codice
Consente
No
0.0.0.0/0
TCP
Tutto
22
Traffico TCP per le porte: 22 SSH Remote

Regole di uscita
Cerca e filtra
Cerca
Aggiungi regole di uscita
Azioni
Senza conservazione dello stato
Destinazione
Protocollo IP
Intervallo porte di origine
Intervallo di porte di destinazione
Tipo e codice
Consente
No
0.0.0.0/0
Tutti i protocolli
Tutto il traffico per tutte le porte

3. Configurazione dell'accesso SSH

3.1 Generazione delle chiavi

Per garantire un accesso sicuro ai managed nodes, le chiavi SSH sono state generate direttamente sul control node. La scelta di generare le chiavi sul control node, piuttosto che utilizzare quelle generate automaticamente dal provider cloud, risponde a una best practice di sicurezza: la chiave privata dovrebbe rimanere sotto il controllo esclusivo dell'amministratore e non essere generata o gestita da infrastrutture di terze parti

E' stato scelto l'algoritmo Ed25519 perche' e' considerato uno dei piu' sicuri e moderni attualmente disponibili per l'autenticazione SSH. Si basa sulla crittografia a curve ellittiche, che offre un livello di sicurezza equivalente a chiavi RSA molto piu' lunghe ma con dimensioni notevolmente ridotte: una chiave Ed25519 a 256 bit offre offre un livello di sicurezza elevato con chiavi di dimensioni ridotte, risultando più efficiente rispetto agli algoritmi RSA tradizionali. Questo si traduce in operazioni crittografiche piu' veloci, firme digitali piu' compatte e una minore superficie di attacco.

Il comando utilizzato per la generazione e':

```
ssh-keygen -t ed25519 -C "ansible-control-node" -f ~/ansible-project/keys/ansible_key

-t ed25519 --> specifica il tipo di algoritmo della chiave
-C "ansible-control-node" --> aggiunge un commento alla chiave (utile per
identificarla)
-f ~/ansible-project/keys/ansible_key --> definisce percorso e nome del file della
chiave (senza estensione per la privata, .pub per la pubblica)
```

```
alberto@ubuntu:~$ mkdir -p ~/ansible-project/keys
alberto@ubuntu:~$ ssh-keygen -t ed25519 -C "ansible-control-node" -f ~/ansible-project/keys/ansible_key
Generating public/private ed25519 key pair.
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /home/alberto/ansible-project/keys/ansible_key
Your public key has been saved in /home/alberto/ansible-project/keys/ansible_key.pub
The key fingerprint is:
SHA256:kKQpixiiK5EeNfIpVehlURnALlymH81babCVG92AcOM ansible-control-node
The key's randomart image is:
+--[ED25519 256]--+
|    o+=oo =Eo    |
|   ..X + * + .   |
|o.++X = + * .    |
|++=Boo = =       |
|*o.oo . S        |
|.oo  . .         |
|o.               |
|.                |
|                 |
+----[SHA256]-----+
alberto@ubuntu:~$ cd ansible-project/keys/
alberto@ubuntu:~/ansible-project/keys$ ls
ansible_key  ansible_key.pub
alberto@ubuntu:~/ansible-project/keys$ cat ansible_key
ansible_key      ansible_key.pub
alberto@ubuntu:~/ansible-project/keys$ cat ansible_key
ansible_key      ansible_key.pub
alberto@ubuntu:~/ansible-project/keys$ cat ansible_key.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAII8bIT69ck9cxmoMC5gusrH142ee4g4ug6SOeaozhtRZ ansible-control-node
alberto@ubuntu:~/ansible-project/keys$
```

Questo comando genera due file: ansible_key (la chiave privata, che resta esclusivamente sul control node) e ansible_key.pub (la chiave pubblica, che viene distribuita ai managed nodes).

3.2 Bootstrap e rotazione delle credenziali

Il processo di accesso ai managed nodes ha simulato uno scenario realistico di provisioning in ambiente cloud:

Fase 1 - Accesso iniziale

Il managed-node-1 è stato creato scegliendo l'opzione di generazione automatica della coppia di chiavi RSA offerta da Oracle Cloud durante il provisioning dell'istanza. La chiave privata è stata scaricata al momento della creazione per consentire il primo accesso.

```
alberto@ubuntu:~$ ssh -i ssh-key-2026-03-28.key ubuntu@92.4.160.80
The authenticity of host '92.4.160.80 (92.4.160.80)' can't be established.
ED25519 key fingerprint is SHA256:sYCY7z2170FShGYNozKCYzIwuJZnTet9o27fdYIzpRM.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '92.4.160.80' (ED25519) to the list of known hosts.
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1044-oracle x86_64)
```

```
ubuntu@managed-node-1:~$ whoami
ubuntu
ubuntu@managed-node-1:~$ hostname
managed-node-1
```

Fase 2 - Rotazione delle credenziali

Una volta effettuato l'accesso con le credenziali temporanee del provider, e' stata aggiunta la chiave pubblica del control node al file authorized_keys del managed-node-1:

```
echo "ssh-ed25519 AAAA...ansible-control-node" >> ~/.ssh/authorized_keys
```

```
ubuntu@managed-node-1:~$ echo "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAII8bIT69ck9cxmoMC5gusrH142ee4g4ug6SOeaozhtRZ ansible-control-node" >> ~/.ssh/authorize
d_keys
ubuntu@managed-node-1:~$
ubuntu@managed-node-1:~$ cat ~/.ssh/authorized_keys
ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAABAQC7Eg1B7OAkQOj5G1W47mrL0/5VhZRkXG+AMOmB25kpLwRSLdt2B8omU83ktMZrIG3IQBslXuel2Nn3SedKOd5AtBc/JTi/SFXYLWlhPlpcWVCJ2aSyN
5iHGNSQzHTHWhobSdg+GOCcHR+hBJKiegpoQhcJTmSHFFynE+UCD8l2yo07qf7pZaoOmMInAgixoDWfe72TS/Ih+dq1i1oof6uTPWgs9IyMvXKRqvhX9UT5sViW+eQ26P6XHjcAxj8NP6OjIa2XzxTz8f
e0Ojd1LMirKgPxnR7A1gu8i8kZd7yz6URXqF9xjNfKUQE08SV2MsHqqpuUqYqxNoAFZRVlHjHp ssh-key-2026-03-28
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAII8bIT69ck9cxmoMC5gusrH142ee4g4ug6SOeaozhtRZ ansible-control-node
ubuntu@managed-node-1:~$
```

Prima di rimuovere la chiave Oracle, e' stato verificato che la nuova chiave funzionasse correttamente aprendo una seconda sessione SSH con la chiave del control node.

```
alberto@ubuntu:~$ ssh -i ~/ansible-project/keys/ansible_key ubuntu@92.4.160.80
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1044-oracle x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
Last login: Tue Apr  7 14:36:09 2026 from 93.44.215.207
ubuntu@managed-node-1:~$ whoami
ubuntu
ubuntu@managed-node-1:~$ hostname
managed-node-1
```

Solo dopo questa verifica, la chiave Oracle e' stata rimossa con il seguente comando:

```
sed -i '/ssh-rsa/d' ~/.ssh/authorized_keys
```

```
ubuntu@managed-node-1:~$ sed -i '/ssh-rsa/d' ~/.ssh/authorized_keys
ubuntu@managed-node-1:~$
ubuntu@managed-node-1:~$ cat ~/.ssh/authorized_keys
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAII8bIT69ck9cxmoMC5gusrH142ee4g4ug6SOeaozhtRZ ansible-control-node
ubuntu@managed-node-1:~$
```

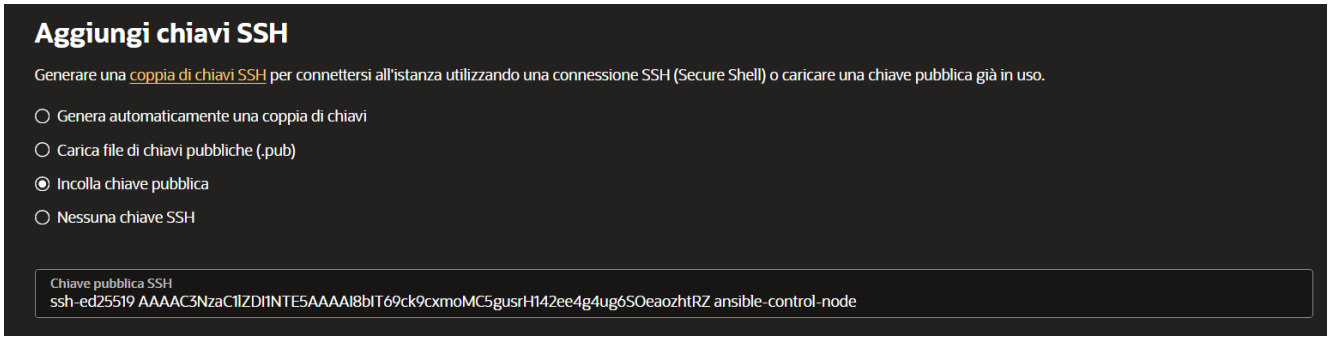
Questo comando utilizza sed per modificare il file authorized_keys direttamente sul posto (opzione -i).
Vengono individuate tutte le righe contenenti la stringa "ssh-rsa" e rimosse dal file. In questo modo, la chiave temporanea del provider viene completamente rimossa e l'unico metodo di accesso rimasto e' la chiave generata dal control node.

Questo approccio prudente evita il rischio di “restare chiusi” fuori dal server: non si rimuove mai una chiave di accesso prima di aver verificato che quella sostitutiva funzioni.

Fase 3 - Managed nodes successivi

Per il managed-node-2, la chiave pubblica del control node e' stata inserita direttamente durante la creazione dell'istanza su Oracle Cloud, tramite l'opzione "Incolla chiave pubblica". In questo modo non e' stato necessario alcun passaggio di rotazione.

Al termine di questo processo, tutti i managed nodes sono accessibili esclusivamente dal control node tramite chiave SSH generata dal control node. Nessun altro metodo di autenticazione e' abilitato.




```
alberto@ubuntu:~$ ssh -i ~/ansible-project/keys/ansible_key ubuntu@130.110.1.219
Welcome to Ubuntu 22.04.5 LTS (GNU/Linux 6.8.0-1044-oracle x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.

*** System restart required ***
Last login: Wed Apr  8 08:49:20 2026 from 93.44.215.207
ubuntu@managed-node-2:~$ whoami
ubuntu
ubuntu@managed-node-2:~$ hostname
managed-node-2
```

4. Installazione e configurazione di Ansible

4.1 Installazione

Ansible e' stato installato sul control node Ubuntu:

```
sudo apt update && sudo apt install -y ansible
```

La versione installata e' Ansible 2.16.3.

La corretta installazione è stata verificata tramite il comando:

```
ansible --version
```

```
alberto@ubuntu:~$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/alberto/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/alberto/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.3 (main, Mar  3 2026, 12:15:18) [GCC 13.3.0] (/usr/bin/python3)
  jinja version = 3.1.2
  libyaml = True
```

4.2 File di configurazione (ansible.cfg)

Il file ansible.cfg definisce i parametri operativi di Ansible e si trova nella directory del progetto ~/ansible-project/. Quando Ansible viene eseguito da una directory contenente un file ansible.cfg, utilizza automaticamente quel file come configurazione.

```
[defaults]
inventory = ./inventory.ini
private_key_file = ./keys/ansible_key
remote_user = ubuntu
```

inventory: specifica il percorso del file di inventario che contiene l'elenco dei managed nodes

private_key_file: indica la chiave privata SSH da utilizzare per l'autenticazione. Corrisponde alla chiave Ed25519 generata in precedenza sul control node

remote_user: definisce l'utente con cui Ansible si connette ai nodi remoti. "ubuntu" e' l'utente predefinito delle istanze Ubuntu su Oracle Cloud

4.3 File di inventario (inventory.ini)

L'inventario definisce i managed nodes. Nel nostro caso:

```
[managed_nodes]
managed-node-1 ansible_host=92.4.160.80
managed-node-2 ansible_host=130.110.1.219
```

Il gruppo [managed_nodes] contiene i due nodi, ciascuno identificato da un nome logico e dal rispettivo indirizzo IP pubblico. Questo nome logico viene utilizzato da Ansible per riferirsi ai nodi nell'output dei comandi e nei playbook.

4.4 Test di connettivita'

Il primo test operativo consiste nel verificare che Ansible riesca a comunicare con tutti i nodi definiti nell'inventario.

Questo viene eseguito con il modulo ping:

```
cd ~/ansible-project
ansible all -m ping
```

Il modulo ping non esegue un ping ICMP tradizionale, ma verifica che Ansible riesca a connettersi via SSH al nodo, che Python sia presente e funzionante, e che il nodo sia in grado di restituire una risposta. Una risposta "pong" indica che la comunicazione funziona correttamente.

```
alberto@ubuntu:~/ansible-project$ ansible all -m ping
managed-node-2 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
managed-node-1 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
```

5. Stato iniziale dei server

Prima di applicare qualsiasi modifica, e' fondamentale documentare lo stato iniziale dei server. Questo consente di avere un riferimento chiaro per il confronto post-hardening e dimostra l'effettivo impatto delle operazioni eseguite dal playbook.

5.1 Comandi di verifica

Le verifiche vengono effettuate tramite comandi ad-hoc di Ansible, che permettono di eseguire singoli comandi su tutti i nodi contemporaneamente:

Configurazione SSH

```
ansible all -a "grep -E 'PermitRootLogin|PasswordAuthentication|MaxAuthTries'
/etc/ssh/sshd_config"

all --> specifica il gruppo di host su cui eseguire il comando (tutti i nodi definiti
nell'inventory)
-a --> indica il comando da eseguire sui nodi remoti
-E --> abilita l'uso di espressioni regolari estese (per utilizzare l'operatore |)
'PermitRootLogin|PasswordAuthentication|MaxAuthTries' --> pattern di ricerca che
individua le principali direttive di sicurezza SSH
/etc/ssh/sshd_config --> file di configurazione del servizio SSH su cui viene
effettuata la verifica
```

```
alberto@ubuntu:~/ansible-project$ ansible all -a "grep -E 'PermitRootLogin|PasswordAuthentication|MaxAuthTries' /etc/ssh/sshd_config"
managed-node-2 | CHANGED | rc=0 >>
#PermitRootLogin prohibit-password
#MaxAuthTries 6
#PasswordAuthentication yes
# PasswordAuthentication.  Depending on your PAM configuration,
# the setting of "PermitRootLogin without-password".
# PAM authentication, then enable this but set PasswordAuthentication
managed-node-1 | CHANGED | rc=0 >>
#PermitRootLogin prohibit-password
#MaxAuthTries 6
#PasswordAuthentication yes
# PasswordAuthentication.  Depending on your PAM configuration,
# the setting of "PermitRootLogin without-password".
# PAM authentication, then enable this but set PasswordAuthentication
```

Il risultato mostra le direttive commentate (precedute da #). In assenza di una configurazione esplicita, il servizio SSH utilizza i valori di default, che in questo caso corrispondono a PermitRootLogin prohibit-password, MaxAuthTries 6 e PasswordAuthentication yes.

Stato del firewall

```
ansible all -b -a "ufw status"
```

```
alberto@ubuntu:~/ansible-project$ ansible all -b -a "ufw status"
managed-node-1 | FAILED | rc=2 >>
[Errno 2] No such file or directory: b'ufw'
managed-node-2 | FAILED | rc=2 >>
[Errno 2] No such file or directory: b'ufw'
```

Il risultato mostra che UFW non e' disponibile sul sistema.
L'opzione -b (become) consente ad Ansible di eseguire i comandi con privilegi elevati (sudo), necessari per interrogare componenti di sistema come il firewall.

Stato di fail2ban

```
ansible all -b -a "systemctl is-active fail2ban"
```

```
alberto@ubuntu:~/ansible-project$ ansible all -b -a "systemctl is-active fail2ban"
managed-node-2 | FAILED | rc=3 >>
inactivenon-zero return code
managed-node-1 | FAILED | rc=3 >>
inactivenon-zero return code
```

Il risultato mostra che il servizio fail2ban non e' attivo ("inactive"), confermando che nessun meccanismo automatico di blocco degli IP dopo tentativi falliti.

5.2 Riepilogo stato iniziale

In sintesi, prima dell'hardening i server presentano:

SSH: configurazione di default, accesso root consentito esclusivamente tramite autenticazione a chiave (PermitRootLogin prohibit-password), autenticazione con password abilitata per gli utenti non root, limite ai tentativi di accesso impostato al valore predefinito

Firewall: assente, tutti i servizi potenzialmente esposti, nessuna restrizione sul traffico di rete

Protezione brute-force: assente, nessun meccanismo automatico di blocco degli IP dopo tentativi falliti

6. Playbook di hardening

6.1 Panoramica

Il playbook di hardening rappresenta il cuore del laboratorio. Si tratta di un file YAML (hardening.yml) che definisce una serie di operazioni di messa in sicurezza da applicare simultaneamente a tutti i managed nodes. Ansible esegue i task in parallelo sui nodi gestiti, questo consente di applicare le configurazioni contemporaneamente su più macchine, riducendo i tempi di esecuzione e garantendo coerenza tra i sistemi.

6.2 Contenuto del playbook

```
---
- name: Hardening dei server --> Nome descrittivo del playbook
  hosts: managed_nodes --> Indica su quali macchine eseguire i task
  become: true --> attiva i privilegi di amministratore (equivalente a sudo)
  tasks:
    - name: Aggiornamento dei pacchetti di sistema
      apt:
        update_cache: yes
        upgrade: dist

    - name: Hardening SSH - Disabilita login root
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PermitRootLogin' --> trova una riga che inizia con PermitRootLogin,
                                         sia che sia commentata (#PermitRootLogin),
                                          sia che non lo sia (PermitRootLogin)
        line: 'PermitRootLogin no'
      notify: Riavvio SSH

    - name: Hardening SSH - Disabilita autenticazione con password
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PasswordAuthentication'
        line: 'PasswordAuthentication no'
      notify: Riavvio SSH

    - name: Hardening SSH - Limita tentativi di autenticazione
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?MaxAuthTries'
        line: 'MaxAuthTries 3'
      notify: Riavvio SSH

    - name: Installazione UFW
      apt:
        name: ufw
        state: present --> Se già installato non fa nulla

    - name: Regola UFW - Consenti solo SSH
      ufw:
        rule: allow
        port: '22'
        proto: tcp
```

```
    - name: Attivazione UFW con policy deny # Attiva il firewall bloccando tutto il
                                              traffico in ingresso tranne SSH
      ufw:
        state: enabled
        default: deny --> Tutto ciò che non è esplicitamente permesso viene bloccato
        direction: incoming

    - name: Installazione fail2ban
      apt:
        name: fail2ban
        state: present

    - name: Crea il file di configurazione personalizzato per fail2ban
      copy:
        content: |
          [sshd]
          enabled = true --> Attiva il monitoraggio del servizio SSH
          port = 22 --> Porta da monitorare
          maxretry = 3
          bantime = 3600
          findtime = 600
          backend = systemd
        dest: /etc/fail2ban/jail.local
        mode: '0644' --> Permessi: leggibile da tutti, scrivibile solo da root
      notify: Riavvio fail2ban --> Se il file viene modificato, riavvia fail2ban

  handlers:
    - name: Riavvio SSH # Viene eseguito solo se notificato da un task SSH
      service:
        name: ssh
        state: restarted

    - name: Riavvio fail2ban # Viene eseguito solo se notificato dal task di
                               configurazione
      service:
        name: fail2ban
        state: restarted
```

6.3 Analisi dettagliata dei task

Intestazione del playbook

La direttiva **hosts: managed_nodes** indica che il playbook verra' eseguito su tutti i nodi del gruppo managed_nodes. La direttiva **become: true** abilita l'escalation dei privilegi (equivalente a sudo), necessaria per modificare file di sistema e gestire servizi.

Aggiornamento dei pacchetti

Il primo task aggiorna la cache dei pacchetti e applica tutti gli aggiornamenti disponibili. Mantenere il sistema aggiornato e' il primo passo fondamentale in qualsiasi procedura di hardening, poiche' le vulnerabilita' note vengono corrette tramite gli aggiornamenti di sicurezza.

Hardening SSH

I tre task successivi modificano la configurazione del servizio SSH agendo sul file /etc/ssh/sshd_config. Ogni task cerca una specifica riga nel file e la sostituisce con il valore desiderato. Se la riga viene effettivamente modificata, Ansible attiva il riavvio del servizio SSH (tramite il meccanismo degli handler) affinche' la nuova configurazione venga applicata. Se la riga contiene gia' il valore corretto, Ansible non fa nulla e il servizio non viene riavviato inutilmente.

PermitRootLogin no: impedisce qualsiasi accesso diretto come utente root via SSH, sia tramite password sia tramite chiave SSH. L'amministrazione del server viene effettuata tramite un utente non privilegiato (nel nostro caso `ubuntu`), con possibilità di escalation dei privilegi tramite `sudo` quando necessario.

PasswordAuthentication no: disabilita completamente l'autenticazione tramite password, obbligando l'utilizzo di chiavi SSH. Questo elimina la possibilita' di attacchi brute-force basati su dizionario di password.

MaxAuthTries 3: limita a 3 il numero di tentativi di autenticazione per sessione. Combinato con fail2ban, questo riduce drasticamente la finestra di opportunita' per un attaccante.

UFW

UFW viene installato e configurato con una policy restrittiva: tutto il traffico in ingresso viene bloccato per impostazione predefinita (deny incoming), con un'unica eccezione per il traffico TCP sulla porta 22 (SSH). Questo garantisce che i server siano raggiungibili solo via SSH e che qualsiasi altro servizio esposto accidentalmente venga automaticamente bloccato dal firewall.

È importante notare che la porta 22 risulta aperta anche a livello di Security List di Oracle Cloud.
Si tratta però di due livelli di protezione distinti e indipendenti:
- la Security List controlla il traffico a livello di rete cloud, prima che arrivi alla macchina,
- UFW lo controlla direttamente sul sistema operativo della macchina stessa.
Questa sovrapposizione è intenzionale e segue il principio della defense in depth: se uno dei due livelli viene modificato o compromesso, l'altro continua a proteggere il server.

Fail2ban

Fail2ban e' un sistema di prevenzione delle intrusioni che monitora i log di sistema alla ricerca di pattern di attacco. La configurazione personalizzata definisce i seguenti parametri per la protezione SSH:

maxretry = 3: dopo 3 tentativi di autenticazione falliti, l'indirizzo IP sorgente viene bannato.

bantime = 3600: la durata del ban e' di 3600 secondi (1 ora).

findtime = 600: i tentativi falliti vengono conteggiati all'interno di una finestra temporale di 600 secondi (10 minuti).

backend = systemd: specifica che fail2ban deve leggere i log dal journal di systemd

6.4 Esecuzione e risultati

Il playbook viene eseguito con il comando:

```
ansible-playbook hardening.yml
```

```
alberto@ubuntu:~/ansible-project$ ansible-playbook hardening.yml

PLAY [Hardening dei server] ************************************

TASK [Gathering Facts] *****************************************
ok: [managed-node-1]
ok: [managed-node-2]

TASK [Aggiornamento dei pacchetti di sistema] ******************
changed: [managed-node-2]
changed: [managed-node-1]

TASK [Hardening SSH - Disabilita login root] *******************
changed: [managed-node-2]
changed: [managed-node-1]

TASK [Hardening SSH - Disabilita autenticazione con password] *****
changed: [managed-node-1]
changed: [managed-node-2]

TASK [Hardening SSH - Limita tentativi di autenticazione] ********
changed: [managed-node-1]
changed: [managed-node-2]

TASK [Installazione UFW] ***************************************
changed: [managed-node-1]
changed: [managed-node-2]

TASK [Regola UFW - Consenti solo SSH] ******************************
changed: [managed-node-1]
changed: [managed-node-2]
```

```
TASK [Attivazione UFW con policy deny] ******************************************************************
changed: [managed-node-1]
changed: [managed-node-2]

TASK [Installazione fail2ban] ***************************************************************************
changed: [managed-node-2]
changed: [managed-node-1]

TASK [Configurazione fail2ban per SSH] ******************************************************************
changed: [managed-node-1]
changed: [managed-node-2]

RUNNING HANDLER [Riavvio SSH] ***************************************************************************
changed: [managed-node-2]
changed: [managed-node-1]

RUNNING HANDLER [Riavvio fail2ban] ******************************************************************
changed: [managed-node-1]
changed: [managed-node-2]

PLAY RECAP ******************************************************************************************
managed-node-1             : ok=12   changed=11   unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
managed-node-2             : ok=12   changed=11   unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

L'output mostra ogni task eseguito su entrambi i nodi, con lo stato "changed" per le modifiche applicate e "ok" per le verifiche senza modifiche. Al termine, il riepilogo (PLAY RECAP) conferma zero errori su tutti i nodi.

7. Verifiche post-hardening

Dopo l'esecuzione del playbook, vengono ripetuti gli stessi comandi di verifica utilizzati per documentare lo stato iniziale, al fine di dimostrare l'effettiva applicazione delle configurazioni.

7.1 Configurazione SSH

```
ansible all -a "grep -E 'PermitRootLogin|PasswordAuthentication|MaxAuthTries'
/etc/ssh/sshd_config"
```

```
alberto@ubuntu:~/ansible-project$ ansible all -a "grep -E 'PermitRootLogin|PasswordAuthentication|MaxAuthTries' /etc/ssh/sshd_config"
managed-node-1 | CHANGED | rc=0 >>
PermitRootLogin no
MaxAuthTries 3
PasswordAuthentication no
# PasswordAuthentication.  Depending on your PAM configuration,
# the setting of "PermitRootLogin without-password".
# PAM authentication, then enable this but set PasswordAuthentication
managed-node-2 | CHANGED | rc=0 >>
PermitRootLogin no
MaxAuthTries 3
PasswordAuthentication no
# PasswordAuthentication.  Depending on your PAM configuration,
# the setting of "PermitRootLogin without-password".
# PAM authentication, then enable this but set PasswordAuthentication
```

Il risultato mostra che le direttive sono ora attive (senza il # di commento) con i valori di hardening: PermitRootLogin no, PasswordAuthentication no, MaxAuthTries 3.

7.2 Stato del firewall

```
ansible all -b -a "ufw status"
```

```
alberto@ubuntu:~/ansible-project$ ansible all -b -a "ufw status"
managed-node-1 | CHANGED | rc=0 >>
Status: active

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW       Anywhere
22/tcp (v6)                ALLOW       Anywhere (v6)
managed-node-2 | CHANGED | rc=0 >>
Status: active

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW       Anywhere
22/tcp (v6)                ALLOW       Anywhere (v6)
```

Il risultato mostra UFW attivo con la sola porta 22/tcp consentita da qualsiasi sorgente (Anywhere). Tutto il restante traffico in ingresso viene bloccato, in quanto la policy di default configurata dal playbook è deny (rifiuta) per le connessioni in entrata.

7.3 Stato di fail2ban

```
ansible all -b -a "systemctl is-active fail2ban"
```

```
alberto@ubuntu:~/ansible-project$ ansible all -b -a "systemctl is-active fail2ban"
managed-node-1 | CHANGED | rc=0 >>
active
managed-node-2 | CHANGED | rc=0 >>
active
```

Il servizio fail2ban risulta attivo ("active").

7.4 Fail2ban in azione

Un risultato particolarmente significativo e' emerso gia' durante le prime fasi di test: fail2ban ha iniziato a bloccare automaticamente indirizzi IP reali che tentavano accessi non autorizzati via SSH ai managed nodes. Si tratta di bot automatici che scansionano costantemente internet alla ricerca di server con SSH esposto, tentando combinazioni di credenziali con attacchi brute-force.

Questo dimostra concretamente l'efficacia della configurazione di hardening: i server sono esposti su internet con un IP pubblico, ma grazie alla combinazione di autenticazione esclusivamente a chiave SSH, firewall restrittivo e sistema di ban automatico, gli attacchi vengono rilevati e bloccati in tempo reale.

```
alberto@ubuntu:~/ansible-project$ ansible all -b -a "fail2ban-client status sshd"
managed-node-1 | CHANGED | rc=0 >>
Status for the jail: sshd
|- Filter
|  |- Currently failed: 4
|  |- Total failed:     10
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 2
   |- Total banned:     2
   `- Banned IP list:   80.66.66.70 101.36.122.139
managed-node-2 | CHANGED | rc=0 >>
Status for the jail: sshd
|- Filter
|  |- Currently failed: 3
|  |- Total failed:     18
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 3
   |- Total banned:     3
   `- Banned IP list:   82.165.66.87 46.37.66.191 185.9.193.111
```

Le verifiche post-hardening dimostrano quindi che tutte le misure di sicurezza sono state applicate correttamente e che l’infrastruttura ha raggiunto uno stato coerente, riproducibile e controllato.

7.5 Idempotenza in pratica

Eseguendo nuovamente il playbook di hardening dopo il primo run, l'output mostra un comportamento differente:
i task di installazione (UFW, fail2ban) restituiscono "ok" perché i pacchetti sono già presenti,
i task di configurazione (SSH, UFW, fail2ban) restituiscono anch'essi "ok" perché lo stato del sistema corrisponde già a quello desiderato.
Nessun handler viene eseguito, poiché non sono state apportate modifiche.
Questo rappresenta il principio di idempotenza in azione: Ansible verifica lo stato attuale del sistema ed esegue operazioni solo quando necessario.

```
alberto@ubuntu:~/ansible-project$ ansible-playbook hardening.yml

PLAY [Hardening dei server] ************************************

TASK [Gathering Facts] *****************************************
ok: [managed-node-2]
ok: [managed-node-1]

TASK [Aggiornamento dei pacchetti di sistema] ******************
ok: [managed-node-2]
ok: [managed-node-1]

TASK [Hardening SSH - Disabilita login root] *******************
ok: [managed-node-2]
ok: [managed-node-1]

TASK [Hardening SSH - Disabilita autenticazione con password] *****
ok: [managed-node-1]
ok: [managed-node-2]

TASK [Hardening SSH - Limita tentativi di autenticazione] ********
ok: [managed-node-1]
ok: [managed-node-2]

TASK [Installazione UFW] ***************************************
ok: [managed-node-1]
ok: [managed-node-2]

TASK [Regola UFW - Consenti solo SSH] ******************************
ok: [managed-node-2]
ok: [managed-node-1]
```

```
TASK [Attivazione UFW con policy deny] *****************************************************************
ok: [managed-node-2]
ok: [managed-node-1]

TASK [Installazione fail2ban] **************************************************************************
ok: [managed-node-1]
ok: [managed-node-2]

TASK [Configurazione fail2ban per SSH] *****************************************************************
ok: [managed-node-2]
ok: [managed-node-1]

PLAY RECAP *********************************************************************************************
managed-node-1             : ok=10   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
managed-node-2             : ok=10   changed=0    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

8. Playbook di reset

8.1 Scopo

Il playbook di reset (reset.yml) riporta i server allo stato precedente all'hardening, consentendo di ripetere la dimostrazione senza dover ricreare le istanze. Il reset non disinstalla i pacchetti installati (UFW e fail2ban), ma si limita a ripristinare le configurazioni modificate.

Questa scelta è intenzionale e permette di evidenziare il principio di idempotenza: alla successiva esecuzione del playbook di hardening, i task di installazione restituiranno "ok" (poiché i pacchetti sono già presenti), mentre i task di configurazione risulteranno "changed", ripristinando le impostazioni di sicurezza.

8.2 Contenuto del playbook

```
---
- name: Reset hardening
  hosts: managed_nodes
  become: true
  tasks:
    - name: Ripristina PermitRootLogin
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PermitRootLogin'
        line: 'PermitRootLogin prohibit-password'
      notify: Riavvio SSH

    - name: Ripristina PasswordAuthentication
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?PasswordAuthentication'
        line: 'PasswordAuthentication yes'
      notify: Riavvio SSH

    - name: Ripristina MaxAuthTries
      lineinfile:
        path: /etc/ssh/sshd_config
        regexp: '^#?MaxAuthTries'
        line: 'MaxAuthTries 6'
      notify: Riavvio SSH

    - name: Disabilita UFW
      ufw:
        state: disabled

    - name: Rimuovi configurazione fail2ban personalizzata
      file:
        path: /etc/fail2ban/jail.local
        state: absent --> Se il file non esiste, non fa nulla

    - name: Arresta e disabilita fail2ban
      service:
        name: fail2ban
        state: stopped --> Arresta il servizio immediatamente
        enabled: false --> Non si avvierà più al riavvio del server
```

```
handlers:
  - name: Riavvio SSH
    service:
      name: ssh
      state: restarted --> Viene eseguito solo se un task SSH ha notificato una
                           modifica
```

```
alberto@ubuntu:~/ansible-project$ ansible-playbook reset.yml

PLAY [Reset hardening] ***************************************

TASK [Gathering Facts] ***************************************
ok: [managed-node-1]
ok: [managed-node-2]

TASK [Ripristina PermitRootLogin] ****************************
changed: [managed-node-2]
changed: [managed-node-1]

TASK [Ripristina PasswordAuthentication] *********************
changed: [managed-node-2]
changed: [managed-node-1]

TASK [Ripristina MaxAuthTries] *******************************
changed: [managed-node-1]
changed: [managed-node-2]

TASK [Disabilita UFW] ****************************************
changed: [managed-node-2]
changed: [managed-node-1]

TASK [Rimuovi configurazione fail2ban personalizzata] ********
changed: [managed-node-1]
changed: [managed-node-2]

TASK [Arresta e disabilita fail2ban] *************************
changed: [managed-node-2]
changed: [managed-node-1]
```

```
TASK [Arresta e disabilita fail2ban] ******************************************************************************
changed: [managed-node-2]
changed: [managed-node-1]

RUNNING HANDLER [Riavvio SSH] *************************************************************************************
changed: [managed-node-1]
changed: [managed-node-2]

PLAY RECAP ********************************************************************************************************
managed-node-1             : ok=8    changed=7    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
managed-node-2             : ok=8    changed=7    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

Il playbook di reset consente quindi di ripristinare rapidamente un ambiente controllato, rendendo possibile ripetere i test e le dimostrazioni in modo coerente e prevedibile.

8.3 Verifiche post Reset

Dopo l'esecuzione del reset, vengono ripetuti gli stessi comandi di verifica per confermare il ritorno allo stato iniziale: le direttive SSH sono tornate ai valori predefiniti (PermitRootLogin prohibit-password, PasswordAuthentication yes, MaxAuthTries 6), UFW risulta disabilitato e fail2ban è inattivo.

```
alberto@ubuntu:~/ansible-project$ ansible all -a "grep -E 'PermitRootLogin|PasswordAuthentication|MaxAuthTries' /etc/ssh/sshd_config"
managed-node-1 | CHANGED | rc=0 >>
PermitRootLogin prohibit-password
MaxAuthTries 6
PasswordAuthentication yes
# PasswordAuthentication.  Depending on your PAM configuration,
# the setting of "PermitRootLogin without-password".
# PAM authentication, then enable this but set PasswordAuthentication
managed-node-2 | CHANGED | rc=0 >>
PermitRootLogin prohibit-password
MaxAuthTries 6
PasswordAuthentication yes
# PasswordAuthentication.  Depending on your PAM configuration,
# the setting of "PermitRootLogin without-password".
# PAM authentication, then enable this but set PasswordAuthentication
```

```
alberto@ubuntu:~/ansible-project$ ansible all -b -a "ufw status"
managed-node-1 | CHANGED | rc=0 >>
Status: inactive
managed-node-2 | CHANGED | rc=0 >>
Status: inactive
```

```
alberto@ubuntu:~/ansible-project$ ansible all -b -a "systemctl is-active fail2ban"
managed-node-2 | FAILED | rc=3 >>
inactivenon-zero return code
managed-node-1 | FAILED | rc=3 >>
inactivenon-zero return code
```

8.4 Re-Hardening

Eseguendo nuovamente il playbook di hardening dopo il reset, i task di installazione risultano "ok" (poiché i pacchetti UFW e fail2ban sono già presenti sul sistema), mentre i task di configurazione risultano "changed", ripristinando le impostazioni di sicurezza.
Questo comportamento conferma sia l'efficacia del playbook di reset sia il principio di idempotenza.
Il ciclo reset --> hardening dimostra inoltre che l'intero processo è ripetibile in modo controllato e prevedibile.

```
alberto@ubuntu:~/ansible-project$ ansible-playbook hardening.yml

PLAY [Hardening dei server] ****************************************

TASK [Gathering Facts] *********************************************
ok: [managed-node-1]
ok: [managed-node-2]

TASK [Aggiornamento dei pacchetti di sistema] **********************
ok: [managed-node-1]
ok: [managed-node-2]

TASK [Hardening SSH - Disabilita login root] ***********************
changed: [managed-node-1]
changed: [managed-node-2]

TASK [Hardening SSH - Disabilita autenticazione con password] ******
changed: [managed-node-1]
changed: [managed-node-2]

TASK [Hardening SSH - Limita tentativi di autenticazione] **********
changed: [managed-node-1]
changed: [managed-node-2]

TASK [Installazione UFW] *******************************************
ok: [managed-node-1]
ok: [managed-node-2]
```

```
TASK [Regola UFW - Consenti solo SSH] ******************************************************************
ok: [managed-node-1]
ok: [managed-node-2]

TASK [Attivazione UFW con policy deny] *****************************************************************
changed: [managed-node-1]
changed: [managed-node-2]

TASK [Installazione fail2ban] **************************************************************************
ok: [managed-node-1]
ok: [managed-node-2]

TASK [Configurazione fail2ban per SSH] *****************************************************************
changed: [managed-node-1]
changed: [managed-node-2]

RUNNING HANDLER [Riavvio SSH] **************************************************************************
changed: [managed-node-1]
changed: [managed-node-2]

RUNNING HANDLER [Riavvio fail2ban] *********************************************************************
changed: [managed-node-1]
changed: [managed-node-2]

PLAY RECAP *********************************************************************************************
managed-node-1             : ok=12   changed=7    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
managed-node-2             : ok=12   changed=7    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

9. Conclusioni

Il progetto ha dimostrato come Ansible permetta di automatizzare in modo efficace e riproducibile le operazioni di hardening su server cloud. Partendo da istanze appena create sul provider, con configurazioni di default e prive di qualsiasi misura di sicurezza aggiuntiva, è stato possibile portarle a un livello di protezione adeguato con l'esecuzione di un singolo comando, applicando le modifiche simultaneamente su tutti i nodi.

L'approccio adottato evidenzia i principali vantaggi dell'automazione infrastrutturale e della gestione della sicurezza come codice (Security as Code), rendendo il processo non solo più veloce, ma anche più affidabile e controllabile rispetto alla configurazione manuale.

I punti chiave emersi dal laboratorio sono:

Coerenza: tutti i server raggiungono lo stesso identico stato di configurazione, eliminando il rischio di discrepanze tipiche della gestione manuale e garantendo una baseline di sicurezza uniforme su tutta l'infrastruttura.

Ripetibilità: il playbook può essere eseguito più volte senza effetti collaterali grazie al principio di idempotenza. Questo lo rende adatto sia all'applicazione iniziale delle configurazioni sia alla verifica periodica dello stato di conformità dei sistemi.

Velocità: l'intero processo di hardening si completa in circa un minuto. Ansible esegue i task in parallelo sui nodi gestiti, rendendo il tempo di esecuzione ridotto e scalabile anche all'aumentare del numero di macchine.

Documentazione intrinseca: il playbook stesso rappresenta una forma di documentazione delle policy di sicurezza applicate. La struttura in YAML è leggibile e autoesplicativa, permettendo di comprendere rapidamente quali configurazioni sono state implementate senza dover analizzare manualmente ogni singolo server.

In conclusione, l'utilizzo di Ansible in questo contesto dimostra come sia possibile gestire in modo centralizzato, automatizzato e sicuro la configurazione di infrastrutture distribuite, riducendo il rischio di errore umano e migliorando significativamente l'efficienza operativa.